Algorithms 1
Homework 10

Nikita Zagainov

December 3, 2024

Доказательства утверждений в графах. Кратчайшие пути. Разные задачи.

1. Для решения этой задачи подходит алгоритм Дейкстры. Добавление константной величины к длине пути в каждой вершине не влияет на результат. Докажем это: при переходе в некоторую вершину, чтобы учесть константную величину, добавляемую к длине пути, можно добавить её к весу ребра, ведущего в эту вершину. А так как эта константа одинакова для всех вершин, то порядок обхода не изменится. Тогда алгоритм Дейкстры даст корректный результат.

   Алгоритм:

```python
import heapq


def dijkstra(
    graph: dict[object, tuple[float, object]], start: object, end: object
) -> tuple[float, list[object]]:
    queue = [(0, [start])]
    visited = set()
    while queue:
        cost, path = heapq.heappop(queue)
        cur_node = path[-1]
        visited.add(cur_node)
        if cur_node == end:
            return cost, path

        for weight, neighbor in graph[cur_node]:
            if neighbor not in visited:
                heapq.heappush(
                    queue,
                    (
                        cost + weight + 3,
                        path + [neighbor],
                    ),
                )
```

   Сложность - $O((|V| + |E|)\log|V|) = O((|V| + |V|k)\log|V|)$, так как количество коридоров по условию не больше $\frac{|V|k}{2}$

2. (a) Предлагаемый алгоритм - алгоритм поиска компонент сильной связности в данном графе:

```python
def inverse_graph(graph: dict[object, list[object]]) -> dict[object, list[object]]:
    reversed_graph = {}

    for node in graph:
        for neighbor in graph[node]:
            if neighbor not in reversed_graph:
```

```python
                reversed_graph[neighbor] = list()
            reversed_graph[neighbor].append(node)

    return reversed_graph


def _dfs(
    graph: dict[object, list[object]],
    node: object,
    visited: set[object],
    start: list[object],
    finish: list[object],
):
    visited.add(node)
    start.append(node)
    finish.append(None)
    for neighbor in graph.get(node, []):
        if neighbor not in visited:
            _dfs(graph, neighbor, visited, start, finish)
    start.append(None)
    finish.append(node)


def dfs(
    graph: dict[object, list[object]],
) -> tuple[list[object | None], list[object | None]]:
    visited = set()
    start = list()
    finish = list()

    for node in graph:
        if node in visited:
            continue
        _dfs(graph, node, visited, start, finish)

    return start, finish


def graph_condensates(graph: dict[object, list[object]]) -> list[list[object]]:
    _, _finish = dfs(inverse_graph(graph))
    order = [node for node in reversed(_finish) if node is not None]

    visited = set()
    condensates = list()

    start = list()
    finish = list()

    for node in order:
        if node in visited:
            continue
        _dfs(graph, node, visited, start, finish)
        condensates.append([n for n in finish if n is not None])
        start.clear()
        finish.clear()

    return condensates
```

Сложность алгоритма — $O(|V| + |E|)$, так как используется инвертирование графа за $O(|V|+|E|)$ и поиск в глубину на инвертированном графе за $O(|V|+|E|)$. Этот алгоритм корректен, так как он находит компоненты сильной связности в графе по построению.

(b) Для решения этой задачи, рассмотрим граф из компонент сильной связности, постро-

енный в предыдущем пункте. этот граф не имеет циклов. Обозначим вершины этого графа как истоки, если к ним не ведёт ни одно ребро из других и как стоки, если из них не ведёт ни одно ребро в другие. Очевидно, что минимальное количество новых рёбер не больше максимума из количества стоков и истоков. Докажем, что количество новых рёбер может равняться этому числу. Полученный после применения поиска КСС граф является множеством подграфов, в которых в сумме n стоков и m стоков. Пока есть несвязные компоненты, проводим рёбра из стоков произвольных подграфов в истоки тех подграфов, если подграфов всего $k \leq \min(n, m)$, то в результате этого действия останется связный граф с $n - k$ стоков и $m - k$ истоков. Потребуется ещё $\max(n, m) - k$ рёбер, чтобы избавится от лишних стоков и истоков.

Алгоритм:

```python
def get_sources_sinks(
    graph: dict[object, set[object]],
) -> tuple[set[object], set[object]]:
    all_nodes = set(graph.keys())
    all_neighbors = set(
        neighbor for neighbors in graph.values() for neighbor in
            neighbors
    )
    sources = all_nodes - all_neighbors
    sinks = all_neighbors - all_nodes

    return sources, sinks


def add_edges_to_connected(
    graph: dict[object, list[object]],
) -> dict[object, list[object]]:
    condensates = graph_condensates(graph)
    if len(condensates) == 1:
        return graph

    node_to_condensate = dict()
    condensate_to_node = dict()
    for idx, condensate in enumerate(condensates):
        for node in condensate:
            node_to_condensate[node] = idx

        condensate_to_node[idx] = node

    condensed_graph = dict()
    for source, neighbors in graph.items():
        source_condensate = node_to_condensate[source]
        for neighbor in neighbors:
            destination_condensate = node_to_condensate[neighbor]
            if (
                source_condensate != destination_condensate
                and destination_condensate
                not in condensed_graph.get(source_condensate, set())
            ):
                if source_condensate not in condensed_graph:
                    condensed_graph[source_condensate] = set()
                condensed_graph[source_condensate].add(
                    destination_condensate
                )

    sources, sinks = get_sources_sinks(condensed_graph)

    while len(sources) > 0 and len(sinks) > 0:
        source = next(iter(sources))
        visited = set()
        _dfs(condensed_graph, source, visited, [], [])
```

```
        possible_sinks = sinks.difference(visited)

        if len(possible_sinks) == 0:
            sink = next(iter(sinks))
        else:
            sink = next(iter(possible_sinks))

        sources.remove(source)
        sinks.remove(sink)

        if condensate_to_node[sink] not in graph:
            graph[condensate_to_node[sink]] = []
        graph[condensate_to_node[sink]].append(
            condensate_to_node[source]
        )

    while len(sinks) > 0:
        sink = next(iter(sinks))
        sinks.remove(sink)
        if condensate_to_node[sink] not in graph:
            graph[condensate_to_node[sink]] = []
        graph[condensate_to_node[sink]].append(
            condensate_to_node[source]
        )

    while len(sources) > 0:
        source = next(iter(sources))
        sources.remove(source)
        if condensate_to_node[sink] not in graph:
            graph[condensate_to_node[sink]] = []
        graph[condensate_to_node[sink]].append(
            condensate_to_node[source]
        )

    return graph
```

Сложность алгоритма - $O(|V|+|E|)$. Первым делом он за $O(|V|+|E|)$ находит конденсаты, далее за $O(|V|+|E|)$ он строит граф конденсатов, а затем за $O(|V|+|E|)$ с помощью нескольких вызовов поиска в глубину, (суммарный размер обхода ограничен суммарным размером подграфов), добавляет новые рёбра. Если же остались неспаренные стоки или истоки, он их добавляет к циклу за $O(|V|)$.

3. Для решения этой задачи можно обойти граф, раскрашивая вершины в два цвета. Если обнаруживается, что две соседние вершины имеют одинаковый цвет, то граф имеет цикл нечётной длины. Если же такого не происходит, то граф не имеет циклов. Докажем это утверждение: если граф не имеет циклов, то этот граф — дерево, и его можно раскрасить в два цвета. Граф, содержащий циклы, может быть получен путём добавления рёбер к дереву. Соединив две вершины дерева, мы получим цикл нечётной длины, если эти вершины имеют одинаковый цвет и цикл чётной длины, если вершины имеют разные цвета. Таким образом утверждение верно по построению.

   Алгоритм:

```
def has_neg_cycle(graph: dict[object, list[object]]) -> bool:
    colors = dict()
    stack = list(graph.keys())

    while stack:
        node = stack.pop()
        if node not in colors:
            colors[node] = 0

        for neighbor in graph.get(node, []):
            if neighbor not in colors:
```

```
                colors[neighbor] = 1 - colors[node]
                stack.append(neighbor)
            elif colors[neighbor] == colors[node]:
                return True

    return False
```

Сложность алгоритма — $O(|V| + |E|)$.

4. Алгоритм обхода в глубину позволяет найти кратчайшие пути в графе с единичными весами рёбер, так как в таком графе порядок обхода вершин будет совпадать с обходом алгоритма Дейкстры. Теперь докажем, что в графе состояний кодового замка для каждой вершины существует другая вершина, расстояние до которой будет равно диаметру графа. Пусть диаметр совпадает с кратчайшем путём между вершинами $abcd$ и $efgh$. Кратчайшим путём между вершинами $abcd$ и $efgh$ будет равен $|a - e| + |b - f| + |c - g| + |d - h|$. тогда повернув кодовый замок на одно деление в произвольной позиции в обоих замках, мы получим две новые вершины, расстояние между которыми будет равно $|(a + 1) \mod 10 - (e + 1) \mod 10| + |b - f| + |c - g| + |d - h| = |a - e| + |b - f| + |c - g| + |d - h|$. Данной операцией мы можем получить любую вершину. Тогда диаметр графа можно найти обходом в ширину из любой вершины.

   Алгоритм:

```
from collections import deque


def switch_lock(lock: str, index: int, direction: int) -> str:
    lock_list = list(lock)
    lock_list[index] = str((int(lock_list[index]) + direction) % 10)
    return "".join(lock_list)


def generate_lock_graph() -> dict[str, list[str]]:
    graph = {}
    for i in range(10000):
        lock_value = f"{i:04d}"
        graph[lock_value] = []
        for j in range(4):
            new_value_1 = switch_lock(lock_value, j, 1)
            new_value_2 = switch_lock(lock_value, j, -1)
            graph[lock_value].append(new_value_1)
            graph[lock_value].append(new_value_2)
    return graph


def get_lock_graph_diameter() -> int:
    graph = generate_lock_graph()
    visited = set()
    queue = deque([(0, "0000")])

    max_distance = 0
    while queue:
        distance, lock = queue.popleft()
        max_distance = max(max_distance, distance)
        visited.add(lock)

        for next_lock in graph[lock]:
            if next_lock in visited:
                continue
            queue.append((distance + 1, next_lock))

    return max_distance
```

Сложность алгоритма — $O(|V| + |E|)$.

5. Для нахождения кратчайшего пути в невзвешенном графе можно воспользоваться алгоритмом BFS. Вызываем его для каждого владельца таксы, чтобы найти ближайшего хозяина таксы для каждого их них.

   Алгоритм:

```python
from collections import deque

def find_closest_dog_masters(
    graph: dict[object, list[object]], dog_owners: set[object]
) -> dict[object, list[object]]:
    if len(dog_owners) == 1:
        return {}

    result = {}
    for dog_owner in dog_owners:
        queue = deque([[dog_owner]])
        visited = set()
        while queue:
            chain = queue.popleft()
            cur_node = chain[-1]
            visited.add(cur_node)
            if cur_node != dog_owner and cur_node in dog_owners:
                result[cur_node] = chain
                break

            for neighbor in graph[cur_node]:
                if neighbor in visited:
                    continue

                queue.append(chain + [neighbor])

    return result
```

   Сложность — $O(m(|V| + |E|))$.